# Five rules for iPRA with ordinary substitution

A raw clause presentation with computational equality

Research report prepared for Vladimir Reshetnikov

November 11, 2026

Comparison snapshot: `fa9d969919315b97796a1dd288f431cd3e41ffca`

**Abstract**

We give a complete proof presentation of full first-order intuitionistic primitive recursive arithmetic using raw scalar-iterator terms, two formula constructors, reusable conditional-universal clauses, and five primitive inference patterns. The numerical domain is unchanged. Both ordinary interpretation composites have internally definable isomorphism graph $x = y$.

The simplification combines the repository's computational equality with its recently submitted clause presentation. It removes canonical term normalization and supplied equality templates from the five-rule interface. The existing successor-seeded proposal also has five rules; being the first five-rule presentation is not a claim of this report. Complete matched Python checkers, proof translations, validation cases, and reproducible source metrics accompany the construction.

The improvement is measured within the transparent, fully validated checker family. The much smaller historical compact encodings are not surpassed. No absolute minimality theorem or new proof-assistant verification is claimed. A separate proved observation shows that uniqueness of zero-starting successor trajectories already implies every zero-equation induction instance; its missing full scalar bootstrap is identified explicitly.

# Contents

**1 Result, scope, and the exact comparison** **3**

**2 A raw clausal calculus with five inference patterns** **5**

**3 Recovering equality, computation, and the logical interface** **6**

**4 Exact translation to and from the raw six-rule calculus** **8**

**5 Ordinary arithmetic and the two interpretation composites** **10**

**6 Complete checkers and verification** **12**

**7 Why a neutral equality does not replace reflexivity** **13**

**8 A narrower arithmetic principle: successor rigidity** **15**

**9 Further research questions** **18**

**Inherited arithmetic development** **19**

**A Complementary introductions with one local assumption** **19**

**B Arithmetic strength and both ordinary interpretation composites** **24**

**C Internal scalar predecessor bootstrap** **26**

**D From literal-equation induction to full intuitionistic PRA** **30**

**E Pairing, primitive recursion, and internal numerical composites** **35**

**F The complete new checker** **38**

# 1 Result, scope, and the exact comparison

The construction is a five-pattern calculus CE5 with the following primitive inventory:

| Reflexivity, Eval, clause use, paired elimination, paired introduction. |
|---|

Its terms are the ordinary finite expressions

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h),$$

where the iterator binds $u$ only in $h$. Its formulas are

$$A ::= (s = t) \mid D_{x,y}A.$$

All substitutions are ordinary capture-avoiding substitutions on raw syntax. Computation is a derivable equation; it is not performed by the term constructor or substitution operation.

The ordinary interpretation of the only logical constructor is

$$D_{x,y}B(x, y) \;:=\; \exists x\big(B(x, 0) \to \forall y\, B(x, y)\big).$$

This is a presentation of full intuitionistic first-order logic coupled to the intended arithmetic. The logical macros include implication, conjunction, disjunction, both quantifiers, and falsity; these are given explicitly in the inherited reference development. There is no use of excluded middle for quantified formulas.

## 1.1 Which current proposal is being improved?

The repository index at the pinned snapshot lists fifty filed research packages [1]. Its two incoming archives must also be considered. In particular, the incoming clause presentation already has five rules [3]. It uses a successor-seeded recursor whose zero case is built into canonical syntax, together with one supplied equality template. The present construction keeps the five-rule count while returning to raw initialized iterators and removing that template.

The resulting tradeoff is explicit:

| Feature | Canonical clause basis | Present raw clause basis |
|---|---|---|
| Primitive inference patterns | Five | Five |
| Formula constructors | Equality and $D$ | Equality and $D$ |
| Arithmetic term constructors | 0 and $R$ | 0, $S$, and $I$ |
| Canonicalization during substitution | Required | None |
| Equality template in a certificate | One term template | None |
| Primitive equality interface | Computation and transport | Reflexivity and Eval |

Variables are understood in both term inventories. Thus the simplification does not reduce every syntactic coordinate: the raw language has an independent successor constructor. Its benefit is a smaller complete matched checker with a more direct substitution specification.

### 1.2 Measured size, including all validation

The comparison programs use the same clause representation, binder-opening convention, closed-goal input contract, validation policy, error handling, and normalization protocol. Every program is a standalone module. Neither a proof compiler nor any imported project code is needed to run one of these checkers.

| Complete program | Normalized bytes | Tokens | AST | Gzip-9 |
|---|---|---|---|---|
| Matched canonical five-rule control | 3,175 | 903 | 1,035 | 1,035 |
| Matched raw six-rule control | 3,252 | 930 | 1,072 | 1,020 |
| **Raw five-rule checker** | 3,046 | 853 | 993 | 990 |
| Upstream canonical checker (reference) | 3,305 | 946 | 1,072 | 1,073 |

Relative to the matched canonical control, the new checker saves 129 normalized bytes (4.1%), 50 significant tokens, and 42 AST nodes. Its normalized gzip-9 file is 45 bytes smaller.

The raw six-pattern control has the two computation equations and one-term crossed transport in place of Reflexivity and Eval. The canonical five-pattern control implements the current successor-seeded clause basis using the same variable encoding and clause interface as the new checker. These are the controlled comparisons. The unchanged upstream canonical implementation is reported separately for orientation.

Metrics are taken after removing Python docstrings and applying `ast.unparse`, followed by one line feed. Significant-token counts exclude formatting, comments, and the end marker. AST nodes are counted by `ast.walk`. Gzip uses compression level 9 and a zero timestamp; it is supplementary information, not the optimization objective. The reproduction script records the exact Python version. For completeness, the readable new source is 3,456 bytes and its gzip-9 file is 1,191 bytes; those figures include its explanatory docstring and original formatting. The normalized measurements above are the controlled source-complexity comparison.

The current compact lower envelope includes a 516-byte checker with a different proof interface and highly compressed implementation [4]. This report does not beat that envelope. It establishes a measured improvement in the transparent fully validated family, together with the mathematical equivalence of the resulting presentation. It does not prove that any particular source string, number of rules, or axiom system is globally minimal. For another reference point, the existing tagged native six-rule computational-equality checker is 3,039 normalized bytes with an open-sequent public interface. Its different representation and contract are not the matched comparison used above; no claim to improve every existing checker is intended.

### 1.3 Attribution and the novelty boundary

The two component ideas are inherited. Refl–Eval is the repository's computational-equality interface [2]; clause contexts and the removal of first-class $Q$ are from the incoming clause presentation [3]. The present contribution is their raw five-pattern combination, the complete matched implementations, and their explicit equivalence and verification package.

The novelty screen checked the current index, the claim summaries of all fifty filed packages (using their root README when no claim summary was present), both incoming reports, and the relevant full specifications. This exact raw clause/Refl–Eval combination was not found there. That is a scoped repository comparison, not a priority claim about the wider literature or a file-by-file review of every archived artifact.

The separate successor-rigidity result in Section 8 is a further mathematical observation. It is deliberately kept separate from the completed presentation: its remaining equality-reflection bootstrap is not assumed anywhere in the proof that CE5 is bi-interpretable with iPRA.

## 2 A raw clausal calculus with five inference patterns

Write CE5 for the following presentation. Numerical terms are raw finite expressions

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only $u$ in $h$ is bound. All substitution is ordinary capture-avoiding substitution. In particular, the terms $I(0, a; u.h)$ and $a$ are distinct syntax, including after substitution. Formulas have just two constructors:

$$F ::= (s = t) \mid D_{x,y}F.$$

The two coordinates of $D$ are bound. All displayed binders may be renamed before a substitution.

A context is a finite reusable list of *clause occurrences*

$$[A; y.B(y)], \qquad y \notin \mathrm{FV}(A).$$

The clause binds $y$ only in its consequent. It is a context object, not a third formula constructor. Put $H = (0 = 0)$. An ordinary formula assumption $F$ abbreviates the constant clause

$$\mathrm{ord}(F) := [H; v.F], \qquad v \notin \mathrm{FV}(F).$$

Thus $\Gamma, F$ below means $\Gamma, \mathrm{ord}(F)$. Clause occurrences, rather than only their values, are retained: a clause-use instruction selects an occurrence in the context.

### 2.1 The partial computational equality operation

For raw numerical terms $s, t, b$, define $\Phi(s, t, b)$ by these three cases, and leave it undefined otherwise:

| $b$ | required literal source | $\Phi(s, t, b)$ |
|---|---|---|
| $s$ | $b \equiv s$ | $t$ |
| $I(0, a; u.h)$ | $s \equiv a$ | $t$ |
| $I(Sn, a; u.h)$ | $s \equiv I(n, a; u.h)$ | $h[t/u]$. |

Here $\equiv$ is literal identity of well-scoped finite syntax, modulo the chosen representation of bound names. The rows are disjoint. In the second row the source is a proper subterm of $b$; in the third row $b$ has one more successor node than its required source. Hence neither row can also satisfy $b \equiv s$. The last two rows have different counter constructors.

This operation performs one specified equality/computation step. It does not normalize terms. Its last row uses equality of the whole preceding iterator value, not equality of an initializer.

### 2.2 Every primitive inference

The complete inventory consists of the following five patterns:

$$\frac{}{\Gamma \vdash s = s}. \tag{Refl}$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash b = r}{\Gamma \vdash r = \Phi(s, t, b)} \quad \text{if } \Phi(s, t, b) \text{ is defined.} \tag{Eval}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \text{if the occurrence } [A; y.B(y)] \text{ is in } \Gamma. \tag{U}$$

$$\frac{\Gamma \vdash D_{x,y}B(x,y) \qquad \Gamma, [B(b,0); y.B(b,y)] \vdash F}{\Gamma \vdash F}. \tag{E}$$

$$\frac{\Gamma, B(t,\eta) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}, \qquad \eta = \begin{cases} z, & B \text{ has equation root and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases} \tag{I}$$

In E, $b$ is fresh for the retained context, the conclusion, and the passive parameters of the matrix. In I, $z$ is fresh for the retained context, witness $t$, and passive matrix parameters. In every instance, the supplied terms and formulas must be well scoped. There is no restriction that $t$ be a numeral.

No further logical, equality, successor, computation, or induction axiom is primitive. In particular, neither a separate hypothesis rule nor a separate $Q$ constructor occurs. The designated contradiction is $0 = S0$. The clause representation and the three cases of $\Phi$ are part of the trusted specification and must be included in a complete checker.

**Lemma 2.1** (Ordinary assumptions and elementary proof operations)**.** *Ordinary assumption use is admissible. So are weakening, injective fresh renaming, and cut for ordinary formula assumptions.*

*Proof.* Refl proves $H$ in every context. U on the selected occurrence $[H; v.F]$, with any numerical target, then proves $F$. Weakening and fresh renaming follow by induction on the finite derivation. Fresh renaming preserves both the equation-root test and the literal-zero test in I.

For ordinary-assumption cut, suppose a distinguished context occurrence is $[H; v.F]$ and a proof of $F$ is supplied without that occurrence. Induct on the derivation that uses it. At a U node selecting the distinguished occurrence, replace the node by the supplied proof, weakened into the local context. At every other node translate its premises recursively. Eigenvariables can first be freshened. This argument uses no arbitrary numerical-substitution theorem. □

## 3 Recovering equality, computation, and the logical interface

The elementary equality constructions below are inherited from the computational-equality proposal [2]; they are repeated to make the present rule inventory independently checkable. They work under arbitrary retained clauses.

**Lemma 3.1** (Equality algebra and both computation equations)**.** *Refl and Eval derive symmetry, transitivity, and*

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

*Proof.* Let $R_s$ denote Refl at $s$, and let $\mathrm{ev}(p, q)$ denote Eval on its two supplied proofs. Its root case is

$$\frac{s = t \qquad s = r}{r = t}.$$

For $p : s = t$, $\mathrm{ev}(R_s, p)$ proves $t = s$. For $p : s = t$ and $q : t = v$, $\mathrm{ev}(q, \mathrm{ev}(R_s, p))$ proves $s = v$.

For zero computation put $A = I(0, a; u.h)$; then $\mathrm{ev}(R_a, R_A)$ proves $A = a$. For successor computation put $C = I(n, a; u.h)$ and $B = I(Sn, a; u.h)$; then $\mathrm{ev}(R_C, R_B)$ proves $B = h[C/u]$. Each computation proof has two reflexivity leaves and one Eval node. □

**Lemma 3.2** (Unrestricted congruence from one iteration)**.** *For every fixed numerical term template $h(u)$, the rule*

$$\frac{\Gamma \vdash s = t}{\Gamma \vdash h[s/u] = h[t/u]}$$

*is admissible. The template may contain any number of occurrences of $u$, including occurrences inside nested iterator bodies.*

*Proof.* For the supplied proof $p : s = t$, set $A_s = I(0, s; u.h)$ and $B_s = I(S0, s; u.h)$. Construct

$$\begin{aligned} z_s &= \mathrm{ev}(R_s, R_{A_s}) && : A_s = s, \\ c_s &= \mathrm{ev}(z_s, R_{B_s}) && : B_s = h[s/u], \\ z_{s,t} &= \mathrm{ev}(p, R_{A_s}) && : A_s = t, \\ \mathrm{Cong}_h(p) &= \mathrm{ev}(z_{s,t}, c_s) && : h[s/u] = h[t/u]. \end{aligned}$$

The first and third lines use the zero row of $\Phi$. The second and fourth use its successor row, whose required source is literally $A_s$. Thus the construction assumes no prior congruence principle. The body substitution performed by Eval supplies all the displayed occurrences of $s$ and $t$.

There are four new Refl leaves and four Eval nodes, and the input proof $p$ occurs once. This is a bound on inference nodes, not on the size of the copied term labels or the serialized certificate. □

**Corollary 3.3** (One-term and equation-template transport)**.** *The following rules are admissible, with no restriction on the source term:*

$$\frac{s = t \qquad h(s) = r}{r = h(t)}, \qquad \frac{s = t \qquad a(s) = b(s)}{a(t) = b(t)}.$$

*Proof.* For the first rule, apply the root case of Eval to the congruence equation from Lemma 3.2 and the second premise. For the second, use the first rule twice:

$$s = t,\ a(s) = b(s) \implies b(s) = a(t) \implies a(t) = b(t).$$

The second use takes the term template $b$. □

**Lemma 3.4** (All case introductions)**.** *For every matrix $B$ and every witness $t$, the rule*

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x, y)}$$

*is admissible, with the usual freshness of $z$.*

*Proof.* The primitive I rule already has this form except when $B$ is an equation and $t$ is literally 0. In that remaining case put $c = I(0, 0; u.0)$. The term $c$ is not literally zero, and zero computation proves $c = 0$.

Under the local assumption $B(c, 0)$, equation-template transport gives $B(0, 0)$. By ordinary-assumption cut insert the supplied branch and obtain $B(0, Sz)$. Symmetry and equation-template transport give $B(c, Sz)$. Primitive I with witness $c$ is in its case mode, and its conclusion is the required $D_{x,y}B(x, y)$. The conclusion has not changed; only the proof's witnessing term has changed. □

**Lemma 3.5** (Transport in every formula)**.** *For each finite Eq/D formula template $F(v)$, from $\Gamma \vdash s = t$ and $\Gamma \vdash F(s)$ one can derive $\Gamma \vdash F(t)$.*

*Proof.* Induct externally on the formula template. Equation templates were handled above. Suppose $F(v) = D_{x,y}B(v, x, y)$. Eliminate the supplied $F(s)$ at a fresh parameter $b$, retaining the clause

$$[B(s, b, 0); y.B(s, b, y)].$$

Introduce $F(t)$ by Lemma 3.4 with witness $b$. Its branch assumes $B(t, b, 0)$. By the induction hypothesis and the symmetry of $s = t$, derive $B(s, b, 0)$. U at $Sz$ gives $B(s, b, Sz)$; the forward induction hypothesis gives $B(t, b, Sz)$. Close the introduction and elimination. Every recursive transport concerns the strictly smaller matrix $B$, so the proof is not circular. □

### 3.1 Conditional closure without a formula constructor

For a fresh unused $w$, abbreviate

$$q_y B(y) := D_{w,y} B(y).$$

The dummy coordinate is a numerical binder, not a new sort or a domain restriction.

**Lemma 3.6** (The three closure interfaces)**.** *The following operations are derivable:*

1. *From* $\Gamma, B(0) \vdash B(Sz)$*, derive* $\Gamma \vdash q_y B$*.*
2. *From* $\Gamma \vdash q_y B$ *and* $\Gamma \vdash B(0)$*, derive* $\Gamma \vdash B(t)$ *for every term* $t$*.*
3. *In any context retaining* $[B(0); y.B(y)]$*, derive* $q_y B$*.*

*Proof.* For the first operation use case introduction with unused witness. For the second eliminate $q_y B$; its fresh witness does not occur in $B$, and the resulting clause is exactly $[B(0); y.B(y)]$. U with the supplied base proves $B(t)$; close E. For the third, under the local case-introduction assumption $B(0)$, use the retained clause at $Sz$, then close the first operation. □

For an equation $E(y)$, primitive I at dummy witness 0 derives $q_y E$ from $\Gamma, E(z) \vdash E(Sz)$. With its base and the second closure interface this is contextual induction for equations. For an equation $E(x, y)$ and any fixed term $t$, the same construction first proves $q_y E(t, y)$ and then, by case introduction at $t$, recovers the older witnessed progress conclusion $D_{x,y} E(x, y)$. Numerical cases for arbitrary formulas come from the first two closure interfaces. These are derived rules, not additional kernel instructions.

## 4 Exact translation to and from the raw six-rule calculus

Let E6 be the raw computational-equality calculus already present in the repository [2]. Its formulas are Eq/$Q$/$D$, its contexts are ordinary retained formulas, and its six patterns are Hyp, Refl, Eval, QE, DE, and the same witness-directed DI. Refl and Eval are exactly the operations above. Hyp selects a formula assumption, and

$$\frac{\Gamma \vdash Q_y B \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)} \quad \text{is QE,}$$

$$\frac{\Gamma \vdash D_{x,y} B \qquad \Gamma, Q_y B(b, y) \vdash F}{\Gamma \vdash F} \quad \text{is DE.}$$

DI has the displayed I conclusion and branch, with ordinary formula assumptions instead of their constant clauses. The same freshness conditions apply.

Define the translation $\tau$ on native formulas by

$$\tau(s = t) = (s = t), \qquad \tau(D_{x,y} B) = D_{x,y} \tau(B), \qquad \tau(Q_y B) = D_{w,y} \tau(B),$$

where each newly inserted $w$ is fresh and unused. Numerical terms are unchanged. Let $j$ include Eq/$D$ formulas into Eq/$Q$/$D$ formulas. The translation $\tau$ preserves free numerical variables, commutes with capture-free numerical substitution up to bound renaming, and preserves the distinction between equation-root and compound formulas.

**Theorem 4.1** (Proof preservation and common-language equivalence)**.** *For every finite native context* $\Gamma$ *and native formula* $A$*,*

$$\Gamma \vdash_{\mathsf{E6}} A \quad \Longrightarrow \quad \mathrm{ord}(\tau\Gamma) \vdash_{\mathsf{CE5}} \tau(A).$$

*For all finite Eq/D contexts* $\Delta$ *and Eq/D formulas* $F$*,*

$$\mathrm{ord}(\Delta) \vdash_{\mathsf{CE5}} F \quad \Longleftrightarrow \quad j\Delta \vdash_{\mathsf{E6}} jF.$$

*The transformations are effective transformations of finite proof trees and preserve their numerical terms.*

*Proof.* For the first statement, induct on the E6 proof. Hyp becomes Refl at 0 followed by U on the appropriate constant clause. Refl and Eval are copied unchanged. QE becomes the second interface of Lemma 3.6. DI is copied with translated matrices and constant local clauses: $\tau$ preserves the matrix's equation/compound root and leaves its numerical witness unchanged, so the chosen mode is exactly the original mode.

For DE, first translate its major premise. E then opens

$$[\,\tau(B)(b,0); y.\tau(B)(b,y)\,].$$

The third interface of Lemma 3.6 derives $q_y\tau(B)(b,y)$ in that context. This is exactly the translated old branch assumption $\tau(Q_yB(b,y))$. Weaken the inductively translated branch to the current context and cut in this proof of its ordinary assumption. Closing E gives the translated conclusion. All inserted dummy coordinates and eigenvariables are freshened first.

For the reverse statement, annotate each retained clause occurrence by its introduction. An initial ordinary clause or an I branch clause $[H; v.G]$ is represented in E6 by the ordinary assumption $jG$. A clause introduced by E is represented by the native assumption $Q_yjB(b,y)$. If equal-looking clause values have different origins, their occurrences retain their respective annotations.

Induct on the CE5 proof under this invariant. A U selecting an ordinary clause translates to Hyp; its proof of $H$ can be omitted. A U selecting an E clause translates to QE on its associated $Q$ assumption and the translated antecedent proof. Refl and Eval are identical. E translates to native DE, and I translates to native DI with the same witness, matrix root, and local ordinary assumption. This proves the reverse implication. The forward implication on the common language is the first statement, since $\tau j$ is the identity. □

**Theorem 4.2** (Both formula composites)**.** *For every Eq/D formula* $F$*,* $\tau jF = F$ *literally, up to immaterial bound renaming. For every native Eq/Q/D formula* $A$*,*

$$A \;\dashv\vdash_{\mathsf{E6}}\; j\tau A.$$

*Both numerical translations are identities.*

*Proof.* The first claim follows from the definition. For the other one, the essential native equivalence is

$$Q_yB(y) \;\dashv\vdash\; D_{w,y}B(y), \qquad w \notin \mathrm{FV}(B).$$

From $Q_yB$, assume $B(0)$, specialize the closure at $Sz$, and use derived case introduction at an unused witness. Conversely, native DE on the right-hand formula opens precisely $Q_yB$; Hyp closes the branch.

This equivalence extends through all enclosing binders. More explicitly, a uniform pointwise interderivability of matrices lifts through $D$ by DE followed by case introduction: convert the target

zero assumption back to the source zero instance, specialize the opened source closure at $Sz$, and convert forward to the target successor instance. It lifts through $Q$ by the same reasoning with an unused witness. These constructions use only the already established equality and case interfaces. External structural induction on $A$ now proves the result. □

These proofs deliberately avoid claiming that whole proofs can be substituted by reusing every rule tag. Although raw $\Phi$ commutes with capture-free numerical substitution, a substitution can change a nonliteral witness into literal 0 and change I's mode. Full substitution admissibility for ordinary retained-formula sequents transfers from the native calculus through the equivalence above. Weakening, fresh renaming and the ordinary cuts used in the proof were established directly.

For arbitrary externally supplied clauses, use the ordinary reading $[A; y.B]^\circ = A^\circ \to \forall y B^\circ(y)$. Its native representation is the corresponding derived implication/universal macro. Clause use then translates to implication elimination followed by universal elimination. The exact common-language theorem does not require external clauses beyond ordinary formula assumptions; it covers every such retained-context consequence, not only closed theorems.

# 5 Ordinary arithmetic and the two interpretation composites

The rule-equivalence theorem is only one part of the requirement. This section states precisely which ordinary first-order theory the calculus presents and why both interpretation composites preserve the whole numerical domain.

## 5.1 An explicit ordinary realization

Let $T_I$ use intuitionistic first-order logic with actual equality and genuine falsity. Its numerical language contains 0, $S$, and every fixed finite scalar-iterator description. Formally, at each stage, for an earlier term $h(u, \vec{p})$ add a function symbol $i_h(n, a, \vec{p})$ with

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

The remaining nonlogical axioms or equivalent contextual rules are

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad 0 = S0 \longrightarrow \bot,$$

and induction for each literal equation $E(n, \vec{p})$, with arbitrary retained assumptions and the usual fresh induction parameter. Only fixed finite descriptions are added. There is no numerical quantification over programs and no total universal evaluator.

The ordinary readings are

$$(s = t)^\circ = (s = t), \qquad (D_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)),$$

$$[A; y.B]^\circ = A^\circ \to \forall y B^\circ.$$

Every primitive rule of CE5 is sound under this reading. Conversely, the equality rules, the ordinary logical rules, coverage, explosion, and equation induction are derived in CE5 by its equivalence with the reference clause calculus. The explicit falsity bridge matters: equational computation alone permits a singleton algebra and must not silently be identified with an inconsistent ordinary theory.

### 5.2 What the inherited scalar bootstrap proves

The appendix supplies the existing internal argument in its necessary dependency order:

1. Construct a scalar predecessor $P$ and prove $P0 = 0$ and $P(Sn) = n$ using equation induction and numerical cases.
2. Construct truncated subtraction and a faithful characteristic of actual numerical equality. The reflection theorem holds at arbitrary elements, including nonstandard elements of a model.
3. Translate each fixed quantifier-free formula to a characteristic equation and obtain its full intuitionistic induction instance.
4. Construct pairing and projections by bounded search, then compile each fixed general primitive recursion to scalar iteration.

The order avoids assuming pairing or a faithful equality detector while constructing predecessor. These constructions are inherited rather than new size-reduction claims of this report.

If $\pi, L, H$ are the resulting pairing and projections, a target recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$$

is compiled as

$$\begin{aligned} V(n, \vec{p}) &= I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))), \\ f^*(n, \vec{p}) &= H(V(n, \vec{p})). \end{aligned}$$

Internal equation induction proves $L(V(n, \vec{p})) = n$, so the projection equations prove both defining equations for $f^*$. This is a finite external compilation on descriptions.

### 5.3 Bi-interpretability, not just agreement on numerals

Interpret a scalar iterator in ordinary iPRA by primitive recursion whose update ignores its counter. Interpret every iPRA description back in $T_I$ by the compiler above. In both directions the domain formula is $x = x$ and numerical equality remains actual equality.

For two fixed iterators with equal seeds and provably equal updates, equation induction proves equality of all their values. The analogous argument proves uniqueness of each ordinary primitive recursion. Induction externally on a finite description now shows that its double translation satisfies its original defining equations and hence is internally equal to the original function. This also checks coherence when substitution changes an update's syntactic description.

Consequently the graph $x = y$ is internally a total bijection that preserves every numerical operation in each composite interpretation. For formulas, the crucial native composite is

$$D_{x,y}B \;\dashv\vdash\; \exists_* x\big(B(x, 0) \Rightarrow_* \forall_* y B(x, y)\big).$$

The forward direction opens its clause, assumes the base, and derives every instance; the backward direction uses the supplied universal in case introduction. The remaining formula composites follow by external structural induction through the explicitly derived logical macros. Thus $T_I$, and therefore the ordinary realization of CE5, is bi-interpretable and equiconsistent with full first-order iPRA.

This is a written mathematical proof using the supplied internal arithmetic development. The executable tests below concern the new checker implementation and finite proof translations; they are not a replacement for the interpretation proof and are not advertised as a new Lean or Rocq formalization.

# 6 Complete checkers and verification

## 6.1 Public contract

Each compared module reads one JSON object with exactly two fields, `goal` and `proof`. The goal must be a closed Eq/$D$ formula. The proof is a finite tagged tree. The process prints `ACCEPT` and returns success exactly when its inference routine returns that goal from the empty context, subject to ordinary finite machine resources. Otherwise it rejects. Unknown tags, extra fields, malformed trees, noninteger indices, scope violations, and escaped eigenparameters are rejected.

The mathematical calculus also has arbitrary retained clauses and open sequents. The public executable interface checks closed theorems, as does the compared incoming clause checker. Arbitrary open consequences can be expressed using the derived implication and universal macros; this does not turn the public JSON contract into an open-sequent API.

## 6.2 A compact but explicit binder representation

The same numerical convention is used in the new checker and both matched controls. Zero is the integer 0. A fresh free parameter is a negative integer level. Positive integers are local bound indices, counting outwards from 1. At free scope $k$ and local binder depth $d$, a scalar integer leaf must lie in the interval $[-k, d]$.

In the raw language, successor is `[0,t]`, iteration is `[n,a,h]`, equality is `[s,t]`, and $D$ is `[B]`. The expected sort distinguishes a successor from an equation. Iteration binds one variable in its third child, and $D$ binds two variables in its child. For $D_{x,y}B$, $y$ is index 1 and $x$ is index 2. This gives one structurally recursive binder-opening function.

Opening replaces the relevant bound index and decreases larger indices by one. All inserted values are closed with respect to free positive indices: their free variables are negative levels. Therefore insertion does not require shifting those values under binders. Their own internal bound indices stay bound to their own constructors. Retained contexts also do not need shifting when a fresh negative level is introduced.

This convention is an implementation choice shared by the controls; it is not credited as a new mathematical simplification. The canonical control additionally erases the body of an iterator whose rebuilt counter is literal zero. The new checker performs no such normalization.

## 6.3 The inference invariant

For every successful call to `derive(p,g,k)`, every retained clause is well scoped at $k$, its antecedent is a formula, its consequent is a one-variable formula template, and the returned value is a well-scoped formula at $k$. Induction on a successful finite inference trace proves this invariant.

Reflexivity validates its term. Clause use validates the index and witness, proves the exact antecedent, and opens the consequent. Eval destructures two successfully derived formulas as equations. A compound formula has one child, so equation destructuring rejects it. A term being unfolded must destructure as a three-child iterator. A nonzero counter must have the successor shape; the validated raw grammar makes its two-child test exact. Hence the shorter structural destructuring does not bypass sort checking.

Paired introduction validates its entire target and witness before opening the body. Its new hypothesis is retained as a constant clause with antecedent $0 = 0$. Paired elimination validates its target before opening the existential witness. In both branches the fresh negative level is outside the scope in which the target was validated, preventing eigenvariable escape. The exact branch result is checked before the parent result is returned.

The canonical and raw computation/transport controls use the same contract and invariant. The comparisons count the full validation and input boundary, not only the small inference kernel.

### 6.4 Executed verification and its limits

The final execution accepted 32 representative positive proofs and verified both raw proof-translation directions with identical end formulas. It rejected 24 adversarial certificates. All 112 public command-line runs, including runs with Python optimization enabled, returned the expected status and output. The canonical control also passed its representative reflexivity, introduction, and canonical-syntax checks.

The accepted corpus includes both iterator computation equations, nested term templates, a used induction hypothesis, clause specialization, compound binder nesting, and capture-avoiding witness insertion. The raw six-to-five translation satisfied the stated bound of at most nine times as many inference nodes on every executed example; this does not bound term-label sizes or JSON length.

The package supplies bidirectional proof transformations for the matched raw equality interfaces. The mathematical derivations justify them for all finite derivations; the recorded executable checks exercise concrete instances, including nesting and parameter binding. Negative examples target violations that could change the theory: unrestricted compound induction, wrong Eval sources, escaped fresh parameters, wrong clause antecedents, and malformed or mistyped public input.

The finite tests establish the stated execution results. They cannot prove that an arbitrary implementation is sound on every possible certificate. The universal equivalence is the written proof in this report; no new proof-assistant verification of these Python modules is asserted.

### 6.5 Reproducing the archive

Use Python 3.12 to run the verification and metric scripts documented in `README.md`. Check a supplied JSON certificate by redirecting it to `python3 code/clausal_eval.py`. Run `latexmk -pdf report.tex` from the archive root to rebuild this document. The report source, its inputs, standalone modules, normalized forms, gzip-9 files, test evidence, and original-source attribution are all included.

## 7 Why a neutral equality does not replace reflexivity

A tempting further reduction is to remove Refl and retain only the neutral equality $H = (0 = 0)$ used by ordinary assumption clauses. The following obstruction shows that this loses even positive-numeral reflexivity.

Let $\mathsf{CE5}^0$ have the same raw terms, formulas, contexts, $\Phi$, and rules U, E, I, and Eval as CE5, but replace the Refl schema by the single axiom $0 = 0$. Its axiom is available in every context.

**Theorem 7.1** (The neutral equality is insufficient)**.** *The weakened calculus* $\mathsf{CE5}^0$ *does not derive* $S0 = S0$ *from the empty context. In particular, it is not consequence-equivalent to* CE5 *on their common raw language.*

*Proof.* Evaluate all numerical terms by their usual operations on the standard natural numbers, including ordinary initialized iteration. However, assign to the *syntactic equation atom* $s = t$ the truth condition

$$[\![s = t]\!]_\nu \quad \Longleftrightarrow \quad ([\![s]\!]_\nu = 0 \text{ and } [\![t]\!]_\nu = 0). \tag{Z}$$

Thus both sides must denote zero. Interpret $D$ and clauses by

$$[\![D_{x,y}B]\!] = \exists x([\![B(x,0)]\!] \to \forall y\,[\![B(x,y)]\!]),$$

$$[\![[A; y.B]]\!] = [\![A]\!] \to \forall y\,[\![B(y)]\!],$$

using ordinary two-valued truth. An empty context is true.

We verify preservation for every primitive rule. The axiom $0 = 0$ satisfies (Z). For Eval, the truth of its two premises implies

$$[\![s]\!] = [\![t]\!] = [\![b]\!] = [\![r]\!] = 0.$$

In the root row of $\Phi$ its value is $t$, so the conclusion $r = t$ satisfies (Z). In the zero-iterator row its value is again $t$, with the same conclusion. In the successor row write $b = I(Sn, a; u.h)$ and $s = I(n, a; u.h)$. The ordinary numerical evaluation satisfies

$$\begin{aligned}[\![h[t/u]]\!] &= [\![h[s/u]]\!] \\ &= [\![I(Sn, a; u.h)]\!] = [\![b]\!] = 0.\end{aligned}$$

The first equality follows because $s$ and $t$ both have ordinary numerical value zero; the second is ordinary iterator computation. Together with $[\![r]\!] = 0$ this makes the Eval conclusion true.

U is valid implication/universal specialization. E is valid existential elimination: its fresh parameter may be assigned the existential witness, and the conclusion and retained context do not depend on that parameter.

For I in case mode, fix an assignment satisfying the retained context and assume the matrix's zero instance at the value of its witness. Validity of the premise gives every successor instance. Every standard natural number is zero or a successor, so all instances hold, proving the desired conditional closure and its existential conclusion.

For I in progress mode, its witness is literally zero. The valid premise says, for every standard $n$, that the truth of $B(0, n)$ implies the truth of $B(0, Sn)$, under the fixed retained context. If $B(0, 0)$ holds, ordinary metatheoretic induction on standard natural numbers gives every instance. If the base does not hold, the conditional closure is already true. This argument is valid for the interpretation (Z); it does not require equation atoms to mean ordinary equality. All fresh-variable conditions ensure that the retained context and witness stay fixed as $n$ varies.

Induction on finite derivations therefore shows that every closed theorem of $\mathsf{CE5}^0$ is true under this interpretation. But $S0$ has ordinary value 1, so (Z) makes $S0 = S0$ false. □

The interpretation in this proof is a counterinterpretation of a syntactic inference system. It is deliberately *not* a first-order arithmetic structure whose equality symbol is interpreted as identity, nor is it the proposed arithmetic interpretation of the new theory. Such a counterinterpretation suffices to establish nonderivability: each permitted inference preserves its truth condition, while the desired conclusion fails it.

If Refl is deleted without even adding $0 = 0$, the still simpler valuation making every equation atom false gives the analogous obstruction.

**Corollary 7.2** (No finite collection of closed equation seeds)**.** *Delete Refl and add any finite collection of closed equation axioms that are true under ordinary standard-natural-number evaluation. Keeping Eval, U, E, and I unchanged still does not recover universal reflexivity.*

*Proof.* Let $A \subseteq \mathbb{N}$ be the finite set of numerical values of the supplied axioms' sides. Both sides of each axiom have the same value, since the axioms were assumed ordinarily true. Replace (Z) by

$$[\![s = t]\!]_\nu \quad \Longleftrightarrow \quad [\![s]\!]_\nu = [\![t]\!]_\nu \in A.$$

Every seed is true. In each Eval row, actual equality of its first premise's values and ordinary iterator computation imply that $\Phi(s, t, b)$ has the ordinary value of $b$. Its second premise then says that this value equals the value of $r$ and belongs to $A$, so the conclusion is true. The previous U/E argument and the case and progress arguments for I apply to this arbitrary atomic predicate on standard $\mathbb{N}$ without change.

Choose $k \notin A$. The closed numeral equation $S^k0 = S^k0$ is false in this rule-preserving interpretation and hence is not derivable. This is a local obstruction for the fixed remaining rules and closed equation seeds; it makes no claim about other axiom forms or different calculi. □

**A retained reflexivity clause is an axiom.** The permanent clause

$$[\, 0 = 0;\ y.\, y = y \,]$$

together with a proof of $0 = 0$ does recover Refl: U at target $t$ gives $t = t$. That clause has the ordinary reading $(0 = 0) \to \forall y(y = y)$ and fails the counterinterpretation above. Initializing a checker with this clause and a neutral equality proof can remove an instruction tag, but it has relocated the missing universal reflexivity axiom into the initialization. Both the axiom and its validation cost must remain in the complete specification. It is not an additional mathematical rule deletion, and it does not justify a four-pattern equivalence claim for the four remaining rules alone.

# 8 A narrower arithmetic principle: successor rigidity

This section gives a proved reduction of an arithmetic principle, not a second completed axiomatization. The existing `scope_kernel` note obtains induction from scalar iterator uniqueness by a retraction argument. The incoming `ipra_scalar_uniqueness` report completes the equality-reflection bootstrap for that unrestricted uniqueness rule. The observation here is narrower: uniqueness just for zero-starting successor trajectories already gives induction for every literal equation with zero on one side. The proof uses one trajectory and does not require an idempotence lemma.

## 8.1 Exact background and the restricted rule

Use ordinary intuitionistic first-order logic with actual equality, the scalar terms 0, $S$, and $I(n, a; u.h)$, and the equations

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The bound variable $u$ occurs only in the body $h$. Retain numerical coverage

$$\forall n\, (n = 0 \vee \exists m\, n = Sm).$$

For the theorem below, neither successor injectivity nor separation of 0 from $S0$ is needed. Define the selector

$$C(v, a, b) := I(v, a; u.b), \qquad C(0, a, b) = a, \quad C(Sv, a, b) = b.$$

The proposed restricted arithmetic rule is

$$\frac{\Gamma \vdash f(0) = 0 \qquad \Gamma \vdash f(Sz) = S(f(z))}{\Gamma \vdash f(t) = t}. \tag{$\mathrm{SR}_0$}$$

Here $f(z)$ is one fixed finite scalar term template; it is not a function variable. The variable $z$ is fresh for $\Gamma$, $t$, and all passive parameters. In particular, the recurrence premise has no local induction hypothesis. This rule is narrower than arbitrary scalar-iterator uniqueness: its seed is zero and its update is always successor. It does have a separate base premise, which must be counted in any implementation comparison.

Literal-equation induction derives ($\mathrm{SR}_0$): induct on $f(n) = n$. The given base supplies the zero case. Under $f(z) = z$, the recurrence gives $f(Sz) = S(f(z)) = Sz$. Unrestricted scalar uniqueness also derives the rule: apply it to $f$ with update $S$ and to the identity template $n \mapsto n$, then use the common zero seed.

### 8.2 Induction by a trajectory that stays inside the predicate

The following sufficient condition makes the proof transparent. Let $E(x)$ be a fixed formula and suppose that the retained context proves its base and its successor step:

$$\Gamma \vdash E(0), \qquad \Gamma, E(z) \vdash E(Sz),$$

where $z$ is fresh. Suppose also that there is a fixed scalar term $e(x)$ such that, uniformly in a fresh $x$,

$$\Gamma \vdash E(e(x)), \qquad \Gamma, E(x) \vdash e(x) = x. \tag{Reset}$$

No decidability of the whole formula $E$ is included in this condition.

Define the finite term

$$r(n) := I(n, 0; u.e(Su)).$$

We first prove $\Gamma \vdash E(r(n))$ using coverage on $n$. If $n = 0$, computation gives $r(n) = 0$, and the base gives the result. If $n = Sm$, computation gives $r(n) = e(S(r(m)))$, and the first (Reset) statement, instantiated at $S(r(m))$, gives the result directly. This argument does not assume $E(r(m))$ in its successor branch and does not use numerical induction.

Now instantiate the given successor step at $r(z)$ and use the already proved $E(r(z))$. It gives $E(S(r(z)))$. The second (Reset) statement consequently yields

$$e(S(r(z))) = S(r(z)).$$

Thus the iterator equations prove, under the unchanged context,

$$r(0) = 0, \qquad r(Sz) = S(r(z)).$$

One application of $\mathrm{SR}_0$ gives $r(t) = t$. Transport the theorem $E(r(t))$ along this equation to obtain $E(t)$.

All substitutions in this proof are capture avoiding. The eigenvariable of $\mathrm{SR}_0$ is chosen fresh for the retained context and passive parameters; the auxiliary witness used when eliminating coverage is also fresh. Consequently the argument works with arbitrary first-order assumptions in $\Gamma$, including quantified assumptions.

### 8.3 Unconditional consequence for zero equations

Take $E(x)$ to be the literal equation $q(x) = 0$ and assume its base and successor step as above. Put

$$e(x) := C(q(x), x, 0).$$

If $q(x) = 0$, selector computation gives $e(x) = x$, which proves the second (Reset) statement. To prove the first, apply coverage to $q(x)$.

If $q(x) = 0$, then $e(x) = x$ and hence $q(e(x)) = 0$. If $q(x) = Sv$, then $e(x) = 0$ and the assumed base $q(0) = 0$ again gives $q(e(x)) = 0$. These are constructive numerical cases. They require neither a test for equality of two arbitrary terms nor separation of zero and successor.

It follows that the stated background plus $\mathrm{SR}_0$ derives the complete contextual rule

$$\frac{\Gamma \vdash q(0) = 0 \qquad \Gamma, q(z) = 0 \vdash q(Sz) = 0}{\Gamma \vdash q(t) = 0}. \tag{ZI}$$

The construction of $e$ and $r$ is effective on finite syntax, so the argument is a uniform finite proof transformation, not only a semantic observation about the standard natural numbers.

### 8.4 The exact unresolved step

This result does not establish full scalar iPRA from $\mathrm{SR}_0$. The missing bootstrap is substantial. A term $\delta(a, b)$ satisfying

$$\delta(a, b) = 0 \quad \longleftrightarrow \quad a = b$$

throughout the numerical domain would transfer ZI to induction on every literal equation. The already established scalar bridge would then give the desired bi-interpretation with iPRA. However, the current predecessor and equality-reflection proofs use inductions on general equations, or uniqueness with updates other than successor. Importing them before establishing this bridge would be circular.

For example, the completed unrestricted-uniqueness predecessor proof compares unbounded trajectories with updates such as $v \mapsto v + 3$, $v \mapsto v + M(v) + 4$, and its factor map $H$. Those are not instances of $\mathrm{SR}_0$. Boolean finite-state invariants can often be translated to ZI, but that alone does not justify those unbounded trajectory equalities. No converse ZI$\Rightarrow\mathrm{SR}_0$, no full equivalence with unrestricted uniqueness, and no smaller complete checker for $\mathrm{SR}_0$ are claimed here.

The precise further research question is whether scalar computation, coverage, separation, and $\mathrm{SR}_0$ construct a faithful equality test without using any unrestricted equality induction. A positive answer would remove the arbitrary update template from the arithmetic uniqueness rule; its two-premise implementation would then need a matched size comparison.

# 9 Further research questions

1. **Can successor rigidity close the arithmetic bootstrap?** The proved reset construction reduces every zero-equation induction instance to $\mathrm{SR}_0$. A faithful scalar equality test would complete the reduction to full equation induction. Constructing it without importing the general inductions being replaced is the substantive missing step.

2. **Can the raw clause interface improve the compact envelope?** The measured benefit here concerns transparent complete checkers. A comparison against the 516-byte profile must charge its public grammar, certificate protocol, all typing checks, and any compilation layer under one explicitly fixed contract. The present measurements do not resolve that question.

3. **Can introduction be made stable under all substitutions at equal or smaller total cost?** The witness-directed rule tests whether the witnessing term is literally zero. Ordinary mathematical consequences are substitution closed through the equivalence proof, but a translated primitive instruction need not remain the identical instruction. The repository's existing guards and changed closure readings address related tradeoffs; a smaller complete stable interface remains a useful target.

4. **Can the exact five-rule equivalence be formalized economically?** The earlier repository contains Lean results for several component reductions. A focused formalization should prove the clause-to-formula translations, the Eval reconstruction of transport, the DI zero-witness case, and both formula composites. A formal checker-refinement proof can then follow the negative-level scope invariant used here.

The immediate completed result is the raw five-pattern presentation with its matched smaller implementation. The questions above identify further mathematical or implementation work and are not premises of its bi-interpretability theorem.

# Inherited arithmetic development

The following five sections reproduce the repository's existing reference development at the pinned snapshot. They are included to make the arithmetic interpretation and its two composites reviewable within this document. Their calculi $\mathcal{C}$ and $\mathcal{N}$ are reference calculi, not additional primitive rules of the five-rule presentation above. The new contribution is their connection to CE5 proved earlier; the predecessor, equality reflection, pairing, and primitive-recursion compiler are inherited mathematics. Original source files and their licence are included unchanged in the archive.

# A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

## A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only $h$ binds the state variable $u$; $n$ and $a$ do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables $x, y$ in a paired formula are distinct. An ordinary assumption $F$ is the constant clause $\langle H; v.F\rangle$, where $v$ is absent from $F$ and $H$ is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \;\; \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F} \;\; \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \;\; \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)} \;\; \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here $\varepsilon$ is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when $B$ has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$

the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter $z$ is fresh for $\Gamma$, the witness $t$, and every fixed parameter of $B$. In DE the eigenparameter $a$ is fresh for $\Gamma$, the conclusion $F$, and every fixed parameter of $B$. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

## A.2 Exact equivalence and the unused-witness construction

**Lemma A.1** (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

*Proof.* Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

**Theorem A.2** (Identity-language equivalence)**.** *For every retained clause context $\Gamma$ and native formula $F$,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

*The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.*

*Proof.* Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness $t$. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in $\pi$ to a fresh $v$. Choose a new variable $u$ absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate $u$ is deliberately unused.

Apply DE to this paired formula. Because $u$ is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at $Sz$ gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness $t$. The original premise proof $\pi$ has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original $t$. The construction instead keeps $t$ as a fixed parameter of $E(t,y)$ and binds only the unused coordinate $u$.

**Proposition A.3** (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of $H$ followed by Use. If an old derivation has $N$ nodes and $r$ atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

*nodes.*

*Proof.* Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of $H$ needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

### A.3 Induction and numerical cases separate cleanly

**Lemma A.4** (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

*for every equation $E$, with the usual freshness condition.*

*Proof.* Choose an unused witness coordinate $u$. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at $t$. Close DE. □

**Lemma A.5** (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

*with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under $\mathsf{D}$ are derivable without using the progress choice.*

*Proof.* For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at $t$ finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness $a$ and introduce the new one by case DI at $a$. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

### A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with $u$ unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where $E$ is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; $u$ is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives $F$.

Given $A$ and $B$, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give $B$ and $A$, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains $x$; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

### A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let $T_{\mathrm{eq}}$ have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh $a$; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary $y$. Introduce the universal, implication, and existential macros and close DE. Backwards,

eliminate the existential at a fresh $a$. Introduce the paired formula with witness $a$ by case DI: the retained base, supplied implication, and universal elimination at $Sz$ give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* y B$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies $T_{\mathrm{eq}}$ with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

### A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

## B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let $T_I$ be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from $T_I$.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of $T_I$. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused $v$: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

## B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply $P$ to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let $\pi, L, H$ be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

**Theorem B.1** (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization $T_I$ and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

*Proof.* Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

# C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [7] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are $0$, $S$, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only $u$. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

## C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of $C$ is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on $b$; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of $N$ is contained in $\{0,1\}$: its zero value is 0, and its successor value is a value of $Z$. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n+2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of $N$; the second follows by one computation. In the third, increasing $n$ adds three to the left argument and therefore applies $Z$ once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing $M$ as remainder modulo three.

## C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next $G$ state is $G(n)+1$; its $M$ value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next $G$ state is $G(n) + 2$; two computations of $M$ change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of $G$.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain $u$ when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v+1$.

### C.3 A comparator from scalar iteration

For every fixed update $h$, equation induction on $n$, keeping $m, a$ and all parameters fixed, proves the semigroup identity

$$I(m+n,a;h) = I(n, I(m,a;h);h).$$

The base is the zero computation and the step applies $h$ to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update $h$, equation induction on $y$, with its initializer $a$ and all parameters of $h$ fixed, proves the rotation identity

$$I(Sy,a;h) = I(y,h(a);h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same $h$ to the previous equality. Here and below $I(y,a;h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y,3a;H) = 3I(y,a;H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y,a;H)) = 3H(I(y,a;H)).$$

The initializer parameter stays $a$ throughout this induction. Equation induction also gives $I(y,1;H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x,1;u.2u), \qquad L(x,y) = N\big(I(y,E(x);H)\big).$$

Then the following entire recursion specification is provable:

$$L(0,y) = 1, \qquad L(Sx,0) = 0, \qquad L(Sx,Sy) = L(x,y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for $H$. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx,Sy) &= N\big(I(Sy,2E(x);H)\big) \\ &= N\big(I(y,H(2E(x));H)\big) \\ &= N\big(I(y,3E(x);H)\big) \\ &= N\big(3I(y,E(x);H)\big) \\ &= N\big(I(y,E(x);H)\big) = L(x,y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus $L$ has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \le y$ is motivation, not a premise of the following argument.

### C.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su,m),u,Su), \qquad W(t,m) = I(t,0;u.B_m(u)).$$

The subscript in $B_m$ records a fixed parameter; $B_m$ is an open term abbreviation, not a function variable.

Numerical cases on $e$ prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on $e$, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on $t$, with $m$ fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed $m$.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (15)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on $t$, with $m$ fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

### C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying $P$ to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

## D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [8]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

### D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let $\mathsf{PRE}$ contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions $P$ and $C$ with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to $\mathsf{PRE}$, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[ (E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{20}$$

Here the schematic expression $E$ is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where $z$ is absent from $\Gamma$ and the passive parameters, discharge the second premise's equation and universally introduce $z$. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in $\Gamma$.

### D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

**Constructor laws.** Successor is injective: applying $P$ to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

**The sign test.** For every $u$,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all $u, v$,

$$C(u,v,v) = v. \tag{27}$$

Both coverage branches compute to $v$, so this identity again needs no induction.

**The countdown test and the notation $W$.** For every $u$,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named $W$ with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every $u$ already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are $Pu$. Thus $K$ and $W$ are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

### D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on $y$. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold $x$ fixed and induct on $y$. At zero both sides are $Sx$. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold $x$ fixed and induct on $y$. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying $P$ to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on $a$ for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

### D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all $x, y$:

$$C(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold $x$ fixed and induct on $y$. At $y = 0$, subtraction gives residual $x$, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at $y$ and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for $r'$.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for $r$. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since $r'$ is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

**Reflection at residual 1.** We have, for arbitrary $a, b$,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying $P$ gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}\big(K(d(Sa,b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a,b) = 0 \ \vee\ e(a,b) = 1. \tag{40}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (25), which is equivalent to $d(Sa,b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict $a$ or $b$ to standard numerals. It also yields decidable equality. In the $e(a,b) = 0$ branch, (39) gives $a = b$. In the $e(a,b) = 1$ branch, an assumption $a = b$ would give $e(a,b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a\, \forall b\, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

### D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{42}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{43}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{44}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of $A$ proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if $A$ held, the already established implication $A \to b_A = 0$ would contradict separation.

**Atomic formulas and falsity.** For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

**Conjunction.** Split $b_A = 0 \vee b_A = 1$. In the zero branch, $A$ holds and $b_{A\wedge D} = b_D$. Its range follows from the range of $b_D$, and its being zero is equivalent to $D$, hence to $A \wedge D$ under the known $A$. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

**Disjunction.** In the branch $b_A = 0$, the formula $A$ holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of $b_D$. If its value is zero, then $D$ and hence $A \vee D$. Conversely, from $A \vee D$, the $A$ alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the $D$ alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

**Implication.** In the branch $b_A = 0$, the formula $A$ holds and $b_{A\to D} = b_D$. Its range follows from the range of $b_D$. If its value is zero, then $D$, which gives $A \to D$. Conversely, $A \to D$ and the known $A$ give $D$, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume $A$; this contradicts $\neg A$, and explosion gives $D$. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

### D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context $\Gamma$ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on $z$. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term $t$. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

**Completion theorem.** In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives $\mathsf{Sep}$. Quantifier-free induction proves $\mathsf{Cov}$: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

## E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [9]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

### E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free

inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter $n$, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments $v$ exactly when $(v+1)^2 \leq n$, and otherwise retains it. QFI in $i$, with $n$ fixed, proves

$$R_n(i) \leq i, \qquad R_n(i)^2 \leq n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \qquad \text{(Root invariant)}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals $i$, and the new state equals $Si$; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \leq n < (q(n)+1)^2. \qquad \text{(Root)}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed $n$ during the induction.

Now define

$$\pi(a, b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \leq k^2 + a < (k+1)^2$ since $a \leq k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of $\pi$ is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

### E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier $g, h$ already have scalar translations $g^*, h^*$. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$
$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every $p_i$ fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for $f^*$ internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

### E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

# F The complete new checker

This is the complete source, including the public input contract, validation, binder opening, all five inference patterns, JSON handling, and process result. The metrics use its docstring-stripped `ast.unparse` normalization. The archive also contains both matched controls and the finite validation scripts.

```
"""Raw clausal iPRA with computational equality.

Terms: 0, negative free levels, positive bound indices, [0,t] (S),
[n,a,h] (I). Formulas: [s,t] (equality), [B] (D).
Contexts contain (antecedent, one-bound-variable body) clauses.
Input: {'goal': a closed formula, 'proof': a finite proof tree}.
"""
import json
import sys


def fail():
    raise ValueError('invalid certificate')


def successor(a):
    return [0, a]


def term(a, scope, depth=0):
    if type(a) is int:
        return -scope <= a <= depth
    return (type(a) is list and len(a) in (2, 3)
            and (len(a) == 3 or a[0] == 0)
            and all(term(t, scope, depth + (len(a) == 3 and i == 2))
                    for i, t in enumerate(a)))


def formula(a, scope, depth=0):
    return (type(a) is list
            and ((len(a) == 2 and all(term(t, scope, depth) for t in a))
                 or (len(a) == 1 and formula(a[0], scope, depth + 2))))


def opening(a, value, depth=0):
    if type(a) is int:
        return value if a == depth + 1 else a - (a > depth + 1)
    size = len(a)
    return [opening(t, value, depth + (2 if size == 1 else int(size == 3 and i == 2)))
            for i, t in enumerate(a)]


def derive(proof, context, scope):
    if type(proof) is not list or not proof or type(proof[0]) is not str:
        fail()
    tag = proof[0]
    if tag == 'u' and len(proof) == 4:
        index, witness, premise = proof[1:]
        if (type(index) is not int or not 0 <= index < len(context)
                or not term(witness, scope)):
            fail()
        antecedent, body = context[index]
        if derive(premise, context, scope) != antecedent:
            fail()
        return opening(body, witness)
    if tag == 'r' and len(proof) == 2:
        a = proof[1]
        if not term(a, scope):
            fail()
```

```
        return [a, a]
    if tag == 'v' and len(proof) == 3:
        s, t = derive(proof[1], context, scope)
        b, r = derive(proof[2], context, scope)
        if b != s:
            c, a, h = b
            if c != 0:
                if len(c) != 2:
                    fail()
                a, t = [c[1], a, h], opening(h, t)
            if a != s:
                fail()
        return [r, t]
    if tag in ('e', 'i') and len(proof) == 4:
        goal, data, branch = proof[1:]
        if not formula(goal, scope):
            fail()
        fresh = -scope - 1
        if tag == 'e':
            paired = derive(data, context, scope)
            if len(paired) != 1:
                fail()
            body = opening(paired[0], fresh, 1)
            clause = (opening(body, 0), body)
            target = goal
        else:
            if len(goal) != 1 or not term(data, scope):
                fail()
            body = opening(goal[0], data, 1)
            eta = fresh if len(goal[0]) == 2 and data == 0 else 0
            clause = ([0, 0], opening(body, eta))
            target = opening(body, successor(fresh))
        if derive(branch, context + [clause], scope + 1) != target:
            fail()
        return goal
    fail()


def verify(data):
    if type(data) is not dict or set(data) != {'goal', 'proof'}:
        fail()
    if not formula(data['goal'], 0):
        fail()
    if derive(data['proof'], [], 0) != data['goal']:
        fail()
    return True


if __name__ == '__main__':
    try:
        verify(json.load(sys.stdin))
    except Exception:
        print('REJECT')
        sys.exit(1)
    print('ACCEPT')
```

# References

[1] ProveIt contributors, *Concise presentations of intuitionistic primitive recursive arithmetic*, README and research index, pinned commit `fa9d969919315b97796a1dd288f431cd3e41ffca`. https://github.com/VladimirReshetnikov/ProveIt/tree/fa9d969919315b97796a1dd288f431cd3e41ffca/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] ProveIt contributors, *Computational equality*, `Research/computational_equality/report.tex`, same snapshot. Refl and Eval, their equivalence with computation and congruence, and the raw six-rule reference calculus.

[3] ProveIt contributors, *Clausal canonical presentation*, `docs/incoming/ipra_clausal_canonical.zip`, especially `paper/five_rules.tex`, same snapshot. Five-rule canonical clause presentation and its formula composites.

[4] ProveIt contributors, *Canonical recursors*, `Research/canonical_recursors/`, same snapshot.

[5] ProveIt contributors, *Scalar uniqueness*, `docs/incoming/ipra_scalar_uniqueness.zip`, same snapshot. Unrestricted scalar-recursion uniqueness and the completed arithmetic bootstrap.

[6] ProveIt contributors, *Complementary introductions with one local assumption*, `Research/certified_zero/reference/inherited_choice_basis.tex`, same snapshot; reproduced in this report.

[7] ProveIt contributors, *Internal scalar predecessor bootstrap*, `Research/certified_zero/reference/scalar_predecessor.tex`, same snapshot; reproduced in this report.

[8] ProveIt contributors, *From literal-equation induction to full intuitionistic PRA*, `Research/certified_zero/reference/equality_reflection.tex`, same snapshot; reproduced in this report.

[9] ProveIt contributors, *Pairing, primitive recursion, and internal numerical composites*, `Research/certified_zero/reference/primitive_recursion.tex`, same snapshot; reproduced in this report.